

ScoutPro — Sports Talent Scouting & Evaluation System

Web Technology Final Project · Ingabire Esther (27202) · Oct 2, 2026 · @PAC CEE

Fit assessment

ScoutPro is a strong fit: it already covers requirements 1, 2 and most of 4, and every remaining requirement has a natural home in the scouting domain. The work is not a new project — it is promoting the five design-only entities to real code and adding the security, messaging and document layers the final brief demands.

#	Requirement	Status after Assignment 4	What to add
1	Problem, users, objectives, quality attributes, user stories	Mostly done	Measurable targets and acceptance criteria (this doc)
2	ER diagram, domain concepts, physical schema	Partly done	Add Team, Sport, Criterion, Shortlist, PhysicalProfile, User/Role; indexes
3	Responsive React/Angular frontend	Not started	React app, 3 breakpoints, wireframes
4	Backend architecture	Done (layered Spring Boot)	Add an event-driven edge via RabbitMQ
5	Relational + non-relational DB	PostgreSQL only	MongoDB for scouting reports, media metadata, audit and notification logs
6	Secure auth incl. OAuth2 + performance	Not started	Google OAuth2 login + JWT, caching, pagination, indexes
7	RabbitMQ for email, SMS, events	Not started	Exchange + 3 queues, email and SMS consumers
8	RBAC	Not started	4 roles, permission matrix, method-level checks

#	Requirement	Status after Assignment 4	What to add
9	Git with PR to main	In use	Feature branches, conventional commits, final PR
Bonus	DevOps, testing, new tech	Postman only	Docker Compose, GitHub Actions, JUnit + Testcontainers

One honest caution: requirements 5 and 7 must solve a real need, not be bolted on. The design below gives MongoDB data that is genuinely document-shaped and gives RabbitMQ work that genuinely should not block an HTTP request.

1. Problem, users, objectives and requirements

Problem statement

Talent scouting in Rwandan and regional sports academies is still recorded on paper, spreadsheets and WhatsApp messages. Evaluations are inconsistent between scouts, history is lost when a scout leaves, and clubs cannot compare athletes fairly. ScoutPro gives scouts one structured, auditable place to register athletes, score them against sport-specific criteria, and surface the strongest talent to clubs.

Target users

Role	Who they are	Main need
Admin	Federation or academy system administrator	Manage users, roles, sports and criteria
Scout	Field evaluator employed by a club or academy	Register athletes and record evaluations quickly, often on a phone
Club Manager	Recruiter or coach at a team	Browse rankings, build shortlists, receive alerts
Athlete	Registered player (or guardian)	View own profile and evaluation history

Objectives

1. Replace ad-hoc scouting records with structured, criterion-based evaluations.
2. Make every evaluation accountable to a named scout.
3. Let clubs discover talent through rankings and shortlists filtered by sport, position and age.
4. Notify the right people by email and SMS when something relevant happens, without slowing the app down.

Scope

In scope: user accounts with OAuth2 login, RBAC, athletes, scouts, teams, sports, criteria, assessments, physical profiles, shortlists, scouting reports with media, email and SMS notifications, responsive web UI, containerised deployment.

Out of scope: payments, contracts, live video streaming, native mobile apps, video storage itself (only links and metadata are stored).

Functional requirements

ID	Requirement
FR1	Users sign in with Google (OAuth2) or email + password; the API issues a JWT
FR2	Admin assigns roles: ADMIN, SCOUT, CLUB_MANAGER, ATHLETE
FR3	Full CRUD on Athlete, Scout, Team, Sport, Criterion
FR4	A scout records an Assessment with one score per Criterion of the athlete's sport
FR5	The overall score is calculated server-side as a weighted average of criterion scores
FR6	Top Talent Ranking filterable by sport, position, age band, with pagination
FR7	Club Managers create Shortlists and add or remove athletes

ID	Requirement
FR8	Scouts attach a free-form Scouting Report (notes, tags, video links) to an assessment
FR9	Email and SMS are sent asynchronously on key events (see section 4)
FR10	Every create, update and delete is written to an audit log

Quality attributes (measurable)

Attribute	Target	How it is verified
Performance	p95 API latency under 300 ms for list endpoints at 50 concurrent users	k6 or JMeter load test
Performance	Ranking page returns in under 500 ms with 10,000 assessments	Seeded DB + load test
Reliability	No notification lost if the email/SMS provider is down; retried up to 3 times, then dead-lettered	Stop the consumer, send events, restart, count delivered
Security	100% of protected endpoints reject requests without a valid JWT (401) or the right role (403)	Automated security tests
Security	Passwords hashed with BCrypt; no secrets in the repository	Code review + git-secrets scan
Usability	A scout records a full assessment in under 2 minutes on a 360 px phone screen	Timed walkthrough with 3 users
Usability	Lighthouse accessibility score of 90 or higher	Lighthouse report
Maintainability	Service-layer test coverage of 70% or higher	JaCoCo report

User stories and acceptance criteria

US1 — Record an assessment. As a Scout, I want to score an athlete against my sport's criteria so that evaluations are consistent.

- Given I am logged in as SCOUT and the athlete is active, when I submit a score (0–100) for every criterion, then the assessment is saved with a calculated overall score and returns 201.
- Given I already assessed this athlete today, when I submit again, then I receive 422 and nothing is saved.
- Given I omit a criterion, then I receive 400 naming the missing criterion.

US2 — Build a shortlist. As a Club Manager, I want to add top-ranked athletes to a named shortlist so that my coaching staff can review them.

- Given I am logged in as CLUB_MANAGER, when I add an athlete to my shortlist, then the athlete appears in it and the athlete receives an email notification within 1 minute.
- Given the athlete is already on that shortlist, then I receive 409.
- Given I am logged in as SCOUT, when I call the shortlist endpoint, then I receive 403.

US3 — Sign in with Google. As any user, I want to sign in with my Google account so that I do not manage another password.

- Given I click Sign in with Google and approve consent, then I am redirected back with a JWT and land on my role's dashboard.
- Given my Google email has no ScoutPro account, then an account is created with the ATHLETE role pending admin approval.
- Given my JWT has expired, then protected requests return 401 and the UI sends me to sign in.

2a & 2b. Domain concepts and ER diagram

The domain has four human actors, three external systems, six core processes and five data objects; the ER diagram below is the relational core that supports them.

Actors. Admin, Scout, Club Manager, Athlete. External: Google (identity provider), an email provider, an SMS provider.

Processes.

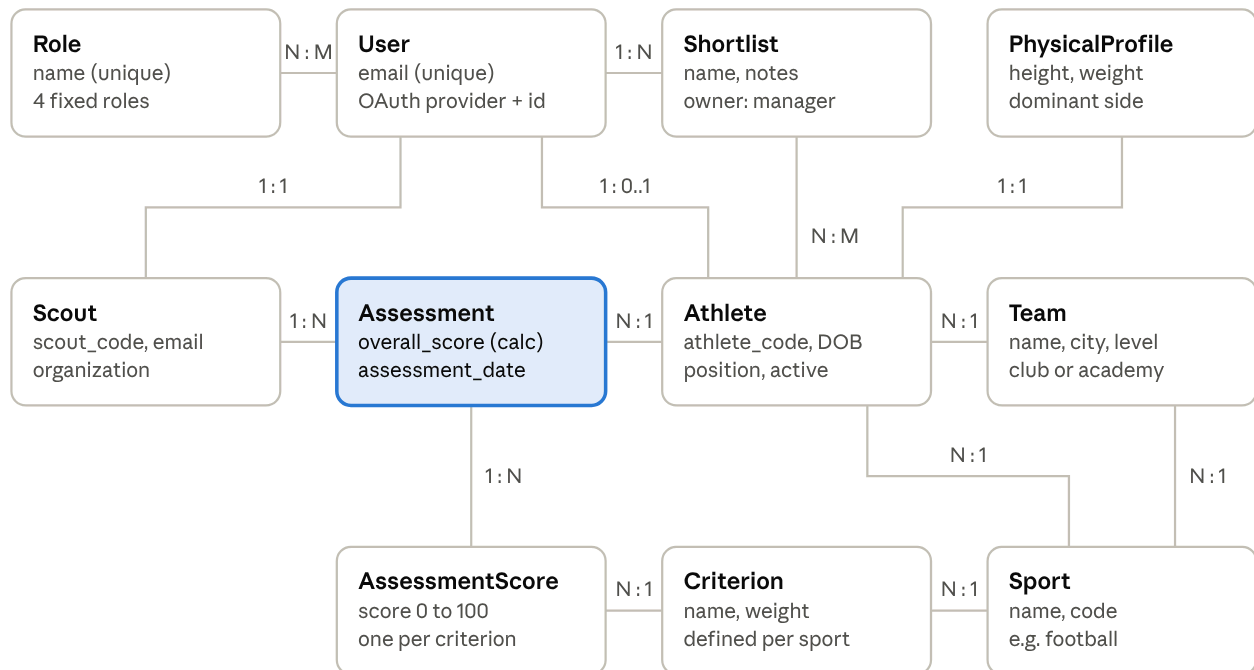
1. Onboarding — a person signs in and an Admin grants a role.
2. Registration — a Scout or Admin registers an athlete and their physical profile.

3. Evaluation — a Scout scores an athlete on every criterion of the athlete's sport and writes a scouting report.
4. Ranking — the system orders evaluations by weighted overall score.
5. Shortlisting — a Club Manager collects promising athletes into named shortlists.
6. Notification — interested parties are told about new evaluations and shortlist decisions.

Data objects. Athlete profile; Evaluation (assessment, criterion scores, scouting report); Sport rulebook (sport and its weighted criteria); Shortlist; Notification and audit records.

Every assessment links one scout to one athlete, scored per criterion

Read labels left to right or top to bottom; N : M pairs use a join table



ScoutPro ER diagram · 11 entities in PostgreSQL

Assessment sits at the centre: it joins a Scout to an Athlete and owns one AssessmentScore per Criterion, which is how scoring becomes sport-specific. Scouting reports, audit logs and notification logs are not drawn here because they live in MongoDB (section 2c & 5).

2c & 5. Physical schema: PostgreSQL + MongoDB

The split rule: **anything with a fixed shape, relationships, or a uniqueness rule lives in PostgreSQL; anything free-form, append-only or high-volume lives in MongoDB.** Every MongoDB document references PostgreSQL rows by UUID, never the reverse, so the relational model stays the source of truth.

PostgreSQL tables

All tables inherit `id UUID PK`, `created_at`, `updated_at` from `BaseEntity`.

Table	Key columns	Foreign keys	Constraints and indexes
users	email, password_hash (nullable for OAuth), provider (LOCAL, GOOGLE), provider_id, enabled	—	UNIQUE(email); UNIQUE(provider, provider_id)
roles	name (ADMIN, SCOUT, CLUB_MANAGER, ATHLETE)	—	UNIQUE(name)
user_roles	user_id, role_id	users.id, roles.id	PK(user_id, role_id)
sports	name, code	—	UNIQUE(code)
criteria	name, weight NUMERIC(4,2)	sport_id → sports.id	UNIQUE(sport_id, name); CHECK(weight > 0)
teams	name, city, level	sport_id → sports.id	UNIQUE(name, sport_id)

Table	Key columns	Foreign keys	Constraints and indexes
athletes	athlete_code, full_name, date_of_birth, position, nationality, contact_number, active	sport_id → sports.id, team_id → teams.id (nullable), user_id → users.id (nullable)	UNIQUE(athlete_code); INDEX(sport_id, position); INDEX(date_of_birth)
physical_profiles	height_cm, weight_kg, dominant_side, measured_at	athlete_id → athletes.id	UNIQUE(athlete_id) — enforces one-to-one
scouts	scout_code, full_name, email, phone_number, organization, active	user_id → users.id	UNIQUE(scout_code); UNIQUE(email); UNIQUE(user_id)
assessments	assessment_code, overall_score NUMERIC(5,2), assessment_date, remarks	athlete_id → athletes.id, scout_id → scouts.id	UNIQUE(assessment_code); UNIQUE(scout_id, athlete_id, assessment_date) — enforces rule 7 in the DB; INDEX(overall_score DESC); INDEX(athlete_id, assessment_date DESC)
assessment_scores	score NUMERIC(5,2)	assessment_id → assessments.id ON DELETE CASCADE, criterion_id → criteria.id	PK(assessment_id, criterion_id); CHECK(score BETWEEN 0 AND 100)
shortlists	name, notes	owner_id → users.id	UNIQUE(owner_id, name)

Table	Key columns	Foreign keys	Constraints and indexes
shortlist_athletes	added_at	shortlist_id → shortlists.id ON DELETE CASCADE, athlete_id → athletes.id	PK(shortlist_id, athlete_id)

Two changes from Assignment 4 worth noting. The three fixed score columns become `assessment_scores` rows, so each sport defines its own criteria (BR8). And the same-day rule moves from service code alone into a database UNIQUE constraint, so it holds even under concurrent requests.

Use **Flyway** migrations (`V1__init.sql` , `V2__criteria.sql` ...) instead of `ddl-auto=update` ; graders and teammates can rebuild the schema exactly.

MongoDB collections

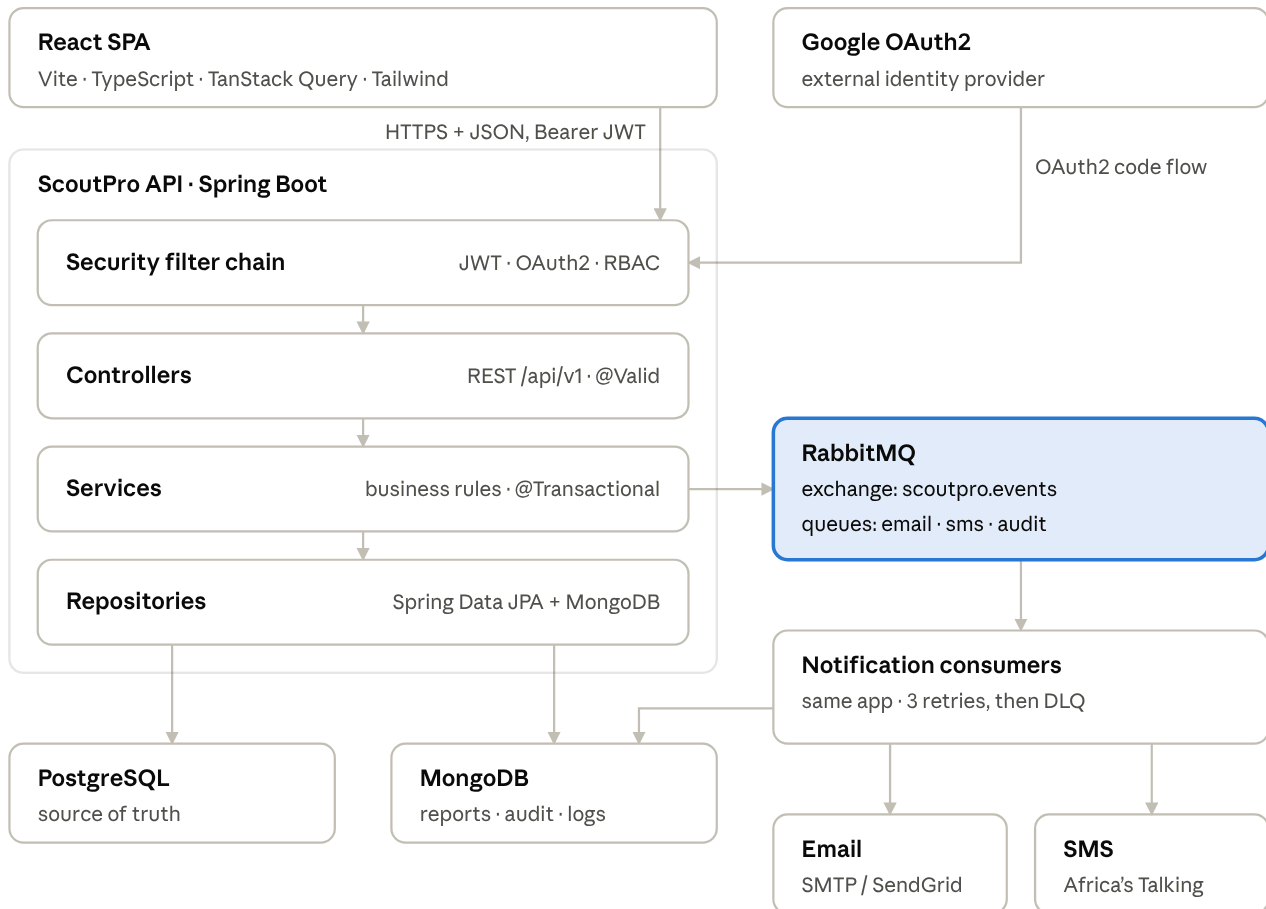
Collection	Why it is a document	Example fields	Indexes
scouting_reports	Free-form notes, variable tags, nested video clips with timestamps — shape differs per sport	assessmentId, athleteId, scoutId, summary, strengths[], weaknesses[], tags[], media[{url, type, startSec, note}]	{athleteId: 1}, text index on summary + tags
audit_logs	Append-only, high volume, never joined	actorId, action, entity, entityId, before{}, after{}, at	{entity: 1, entityId: 1}, TTL 365 days on at
notification_logs	One record per email/SMS attempt with provider response	eventId, channel, recipient, status, attempts, providerResponse{}, sentAt	{eventId: 1} unique — makes consumers idempotent

Access MongoDB with Spring Data MongoDB (`MongoRepository`) beside the existing JPA repositories.

4, 6, 7 & 8. Architecture, security, messaging and RBAC

Keep the layered Spring Boot API from Assignment 4 and add one event-driven edge: services publish domain events to RabbitMQ, and @RabbitListener consumers in the same Spring Boot app send email and SMS. This gives you two architecture styles to defend (layered + event-driven) in one deployable; the consumers sit in their own notification package so they could be split into a separate service later.

Requests stay synchronous; email and SMS leave through RabbitMQ



ScoutPro architecture · layered API + event-driven notifications

The user never waits for an email provider: the API commits the transaction, publishes the event and returns; delivery happens in the background and is retried on failure.

Authentication (requirement 6)

1. The React app sends the user to `/oauth2/authorization/google` (Spring Security OAuth2 Client).

2. Google returns an authorization code; Spring exchanges it for the user's profile.
3. A success handler finds or creates the `users` row, then issues a short-lived **access JWT (15 min)** and a **refresh token (7 days, stored hashed)**.
4. Every later request carries `Authorization: Bearer <jwt>`; the API acts as an OAuth2 Resource Server and validates the signature and expiry.
5. Email + password login is kept as a fallback, with BCrypt hashing.

Authorization: RBAC matrix (requirement 8)

Enforced with `@PreAuthorize` on service methods, plus ownership checks (a scout edits only their own assessments; an athlete reads only their own profile).

Resource / action	ADMIN	SCOUT	CLUB_MANAGER	ATHLETE
Users and roles: manage	Yes	No	No	No
Sports and criteria: create, edit	Yes	No	No	No
Athletes: create, edit	Yes	Yes	No	Own contact info
Athletes: read	Yes	Yes	Yes	Own only
Assessments: create	No	Yes	No	No
Assessments: edit, delete	Yes	Own only	No	No
Ranking: read	Yes	Yes	Yes	No
Shortlists: manage	Yes	No	Own only	No
Scouting reports: read	Yes	Yes	Yes	Own, summary only

Unauthenticated requests return 401; authenticated users without permission get 403, mapped in the existing `GlobalExceptionHandler`.

RabbitMQ events (requirement 7)

One topic exchange `scoutpro.events`; each queue has a dead-letter queue for messages that fail 3 times.

Routing key	Published when	Consumers	Effect
user.registered	A new account is created	email	Welcome email; admin alerted to approve role
assessment.created	A scout saves an assessment	email, audit	Athlete (or guardian) emailed a summary
shortlist.athlete-added	A manager shortlists an athlete	email, sms	Athlete receives email + SMS
athlete.deactivated	Admin deactivates an athlete	email, audit	Scouts who assessed them are informed
entity.changed	Any create, update, delete	audit	Written to MongoDB <code>audit_logs</code>

Publish events **after** the transaction commits (`@TransactionalEventListener (phase = AFTER_COMMIT)`), so a rolled-back assessment never triggers an email. Consumers check `notification_logs.eventId` first, so a redelivered message is not sent twice.

Performance (requirement 6)

- Pagination and sorting on every list endpoint (`Pageable`) instead of returning all rows.
- Database indexes from section 2c, especially `overall_score DESC` for ranking.
- Cache the ranking and sports/criteria lookups in Redis through Spring Cache (`@Cacheable`), evicted on `assessment.created` . Redis is also your bonus new technology.
- Fix N+1 queries with `@EntityGraph` or join fetch on assessment lists; return DTOs, not entities.
- Gzip responses; on the frontend, code-split routes and let TanStack Query cache server data.

3. Frontend: pages and responsive design

Proposed stack: **React + TypeScript (Vite)**, React Router, TanStack Query for server state and caching, Tailwind CSS for responsive layout, React Hook Form + Zod for validation that mirrors the backend rules. Wireframes in Figma at three breakpoints: mobile 360 px, tablet 768 px, desktop 1280 px.

Page	Roles	Mobile layout (360 px)	Desktop layout (1280 px)
Login	All	Single column, Google button first	Split screen: brand panel + form
Dashboard	All (content per role)	Stacked stat cards, bottom nav	Sidebar nav, 4-column stat grid
Athletes list	Admin, Scout, Club Manager	Cards with search bar, filters in a drawer	Table with inline filters and pagination
Athlete profile	All (athlete sees own only)	Tabs: Info, Physical, History	Two columns: profile left, assessment history chart right
New assessment	Scout	One criterion per row with large sliders, sticky Save button	Form left, live overall-score preview right
Top Talent Ranking	Scout, Club Manager	Ranked cards, filter chips	Table with sport/position/age filters, Add to shortlist action
Shortlists	Club Manager	List of shortlists, swipe to remove athlete	Board: shortlist columns side by side
Scouting report	Scout, Club Manager	Text + embedded video links stacked	Report beside the assessment scores
Admin: users, sports, criteria	Admin	Lists with edit sheet	Tables with modal editors

The New assessment page is the one to design mobile-first, because scouts use it at the pitch side. The UI hides actions a role cannot perform, but the backend still enforces every rule — the UI is convenience, not security.

9 & bonus. Git, testing, DevOps and roadmap

Git workflow

- One repository with `backend/`, `frontend/`, `docker-compose.yml` and `docs/` (or two repos if your instructor prefers — confirm).

- Long-lived `main` and `develop`; one `feature/<short-name>` branch per feature, merged into `develop` by pull request.
- Conventional commits: `feat(assessment): score per criterion`, `fix(auth): refresh expired JWT`, `test(shortlist): 403 for scouts`.
- Final deliverable: one pull request `develop → main` with a description linking this document, the Postman collection and the demo video.

Testing

Level	Tool	What it proves
Unit	JUnit 5 + Mockito	Service business rules (age, same-day, score calculation)
Integration	Spring Boot Test + Testcontainers (PostgreSQL, MongoDB, RabbitMQ)	Repositories, queues and real SQL constraints
Security	spring-security-test (<code>@WithMockUser</code>)	Every endpoint returns 401/403 correctly per role
API	Postman collection + Newman in CI	The contract from Assignment 4 still holds
Frontend	Vitest + React Testing Library	Forms, validation messages, role-based rendering
Load	k6	The performance targets in section 1

DevOps

- **Docker Compose** brings up backend, frontend (nginx), PostgreSQL, MongoDB, Redis and RabbitMQ (with management UI) in one command.
- **GitHub Actions**: on every PR run build, tests and Newman; on merge to `main` build and push Docker images to GitHub Container Registry.
- Optional deploy target: a single VPS or Render/Railway; secrets via environment variables, never committed.

Delivery roadmap

1. Week 1 — Requirements and models finalised (this doc), Figma wireframes, Flyway migrations for all tables.

2. Week 2 — Security: users, roles, JWT, Google OAuth2, RBAC on existing endpoints.
3. Week 3 — New entities: Sport, Criterion, Team, PhysicalProfile, Shortlist; per-criterion scoring.
4. Week 4 — MongoDB (reports, audit) and RabbitMQ (email, SMS consumers with retry and DLQ).
5. Week 5 — React frontend, all pages, responsive.
6. Week 6 — Testing, caching and performance, Docker Compose, CI, final PR and demo video.